

ПО ДЗ волк, коза, капуста: 3 способа решить задачу

➤ `never{`

`do`

`:: (fin & gc & wg) -> break;`

`:: else`

`od`

`}`

➤ `ltl {!(<> fin && [] ( wg && gc ) ) }`

➤ `active proctype monitor()`

`{atomic { (fin & gc & wg) -> assert(false) ;}}`

Нужно было выбрать 1 из первых двух способов (работаем в режиме верификации). Затем открыть сгенерированный контрпример и проанализировать траекторию

Задача об обедающих философах. Обнаружение ситуации общего голодания (взаимная блокировка процессов)



Философы – процессы

Вилки – общие ресурсы

Блокировка процессов, конкурирующих за разделяемые ресурсы

- образное описание проблемы, возникающие при совместном использовании ресурсов несколькими параллельными процессами
- за круглым столом сидят несколько философов, каждый из которых проводит время в размышлениях, изредка прерываясь на еду
- на столе стоит блюдо со спагетти и вилки, по одной между каждыми двумя философами
- чтобы поест, каждый философ должен использовать две вилки, которые лежат непосредственно слева и справа от него
- философ берет вилки, если они не заняты, ест спагетти, после чего возвращает вилки на место
- если правая либо левая вилка занята соседом, философ просто ждет ее освобождения

О ситуации общего голодания

- философы строго следуют своим правилам
- блокировка процессов, конкурирующих за разделяемые ресурсы, возникает очень редко, но она приводит к остановке всех процессов
- может сложиться условие, при котором все философы, умрут голодной смертью даже при наличии полного блюда спагетти

Возможные состояния философа

S_0 - философ размышляет. В этом состоянии он может проголодаться – тогда спонтанный переход в состояние S_1

S_1 - философ хочет есть, поэтому ему требуются вилки. Философ тянется за вилкой, лежащей слева от него. Если эта вилка отсутствует, то он в состоянии S_1 ждет ее освобождения. Если левая вилка свободна, то он ее берет и переходит в состояние S_2

S_2 - философ ожидает правую вилку. Если правая вилка отсутствует, то он будет находиться в этом состоянии до ее освобождения. Если правая вилка свободна, то он ее берет и переходит в состояние S_3

S_3 - философ ест.

S_4 - После того, как философ поел, он кладет на стол левую вилку, переходя в следующее состояние S_5 .

S_5 - философ кладет на стол правую вилку и возвращается к размышлениям (в состояние S_0)

Возможные состояния каждой вилки

$F0$ - вилка свободна. Она лежит на столе и готова к тому, что ее возьмет один из философов (слева или справа). Если ее берет левый философ, она переходит в состояние $F1$, если ее берет правый философ, она переходит в состояние $F2$

$F1$ - Вилка находится у философа слева и ожидает возвращения на стол (перехода в состояние $F0$)

$F2$ - Вилка находится у философа справа и ожидает возвращения на стол (перехода в состояние $F0$)

Описание модели обедающих философов на языке Promela

Решаем задачу для N философов

Введем в модель $2N$ процессов: N процессов потребуется для описания поведения философов и N процессов – для описания поведения вилок

Состояние философа фиксируем в переменной *cur_state*

Взятию вилки философом и ее возвращению (освобождению) сопоставим передачу сообщения по рандеву-каналу от философа к вилке

Всего вводится $2N$ каналов:

```
chan l_fork[N] = [0] of {bit}; /* Для левой к философу вилки */  
chan r_fork[N] = [0] of {bit}; /* Для правой к философу вилки */
```

Описание модели (продолжение)

Каждому философу необходимо два канала – один для организации обмена с левой вилкой, другой – для организации обмена с правой вилкой

Использование рандеву-каналов гарантирует синхронизацию взаимодействия между философом и вилкой: философ получит вилку только тогда, когда вилка будет свободна; лежащая на столе вилка поступит в распоряжение философа только тогда, когда философ ее запросит

Для передачи запроса о доступе к вилке и об ее освобождении будет достаточно сообщений одного типа, которые фактически будут играть роль сигнала для вилки и философа для изменения состояния:

```
#define msgtype 1 /* тип сообщения */
```

Процесс «философ»

```
proctype phil (chan left, right; byte mn)
{
    byte cur_state = 0;
    printf("MSC: phil # %d \n", mn);
    do
        :: (cur_state == 1) -> left ! msgtype ->
            cur_state = 2;
        :: (cur_state == 2) ->
            right ! msgtype;
            cur_state = 3;
        :: (cur_state == 3) ->
            cur_state = 4;
        :: (cur_state == 4) ->
            right ! msgtype ->
            cur_state = 5;
        :: (cur_state == 5) ->
            left ! msgtype ->
            cur_state = 0;
        :: (cur_state == 0) -> cur_state = 1;
    od }
```


Описание модели (продолжение)

Для хранения состояния процесса-вилки используется переменная `cur_state_fork`

Взаимодействие с философами осуществляется по тем же рандеву-каналам – по одному каналу на каждого философа (один канал для левого философа, один для правого)

Когда вилка лежит на столе (она свободна), то ожидается запрос либо от левого, либо от правого философа на ее использование. Когда один из философов взял вилку, то по соответствующему каналу ожидается сигнал о возвращении вилки

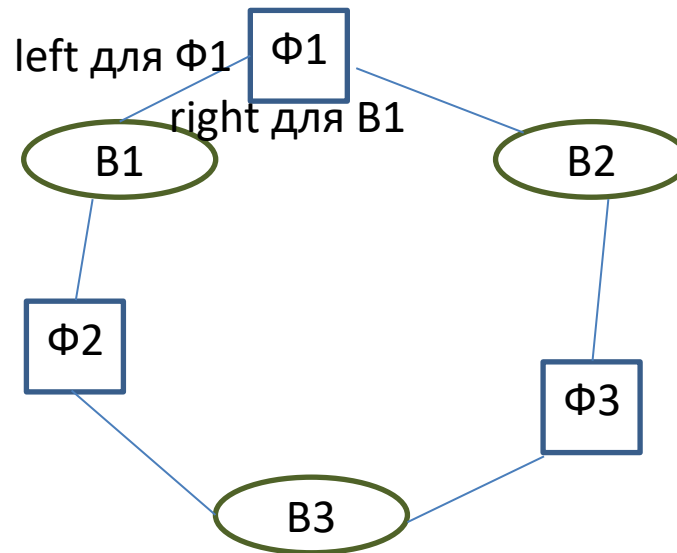
Процесс «вилка»

```
proctype fork(chan left_phil, right_phil)
{
    byte cur_state_fork = 0;
end:  do
    ::( cur_state_fork == 0) ->
        if
            ::right_phil ? msgtype -> cur_state_fork = 2;
            ::left_phil ? msgtype ->  cur_state_fork = 1;
            ::skip;
        fi
    ::( cur_state_fork == 1) ->
        left_phil ? msgtype -> cur_state_fork = 0;
    ::( cur_state_fork == 2) ->
        right_phil ? msgtype ->  cur_state_fork = 0;

od

}
```

Моделирование задачи об обедающих философах при наличии каналов



N процессов-философов

N процессов-вилок

N рандеву-каналов для левых (по отношению к философам) вилок

N рандеву-каналов для правых (по отношению к философам) вилок

Нумерация процессов, вилок, каналов

N процессов-философов и N
процессов-вилок с номерами от 1 до N

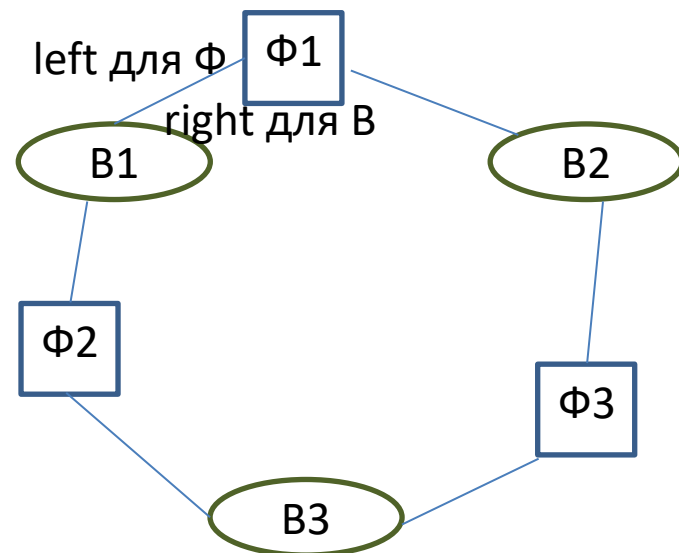
N каналов для левых/правых по
отношению к философам вилок с
номерами от 0 до N-1

Если процесс-философ имеет номер
proc, то proc его правая вилка

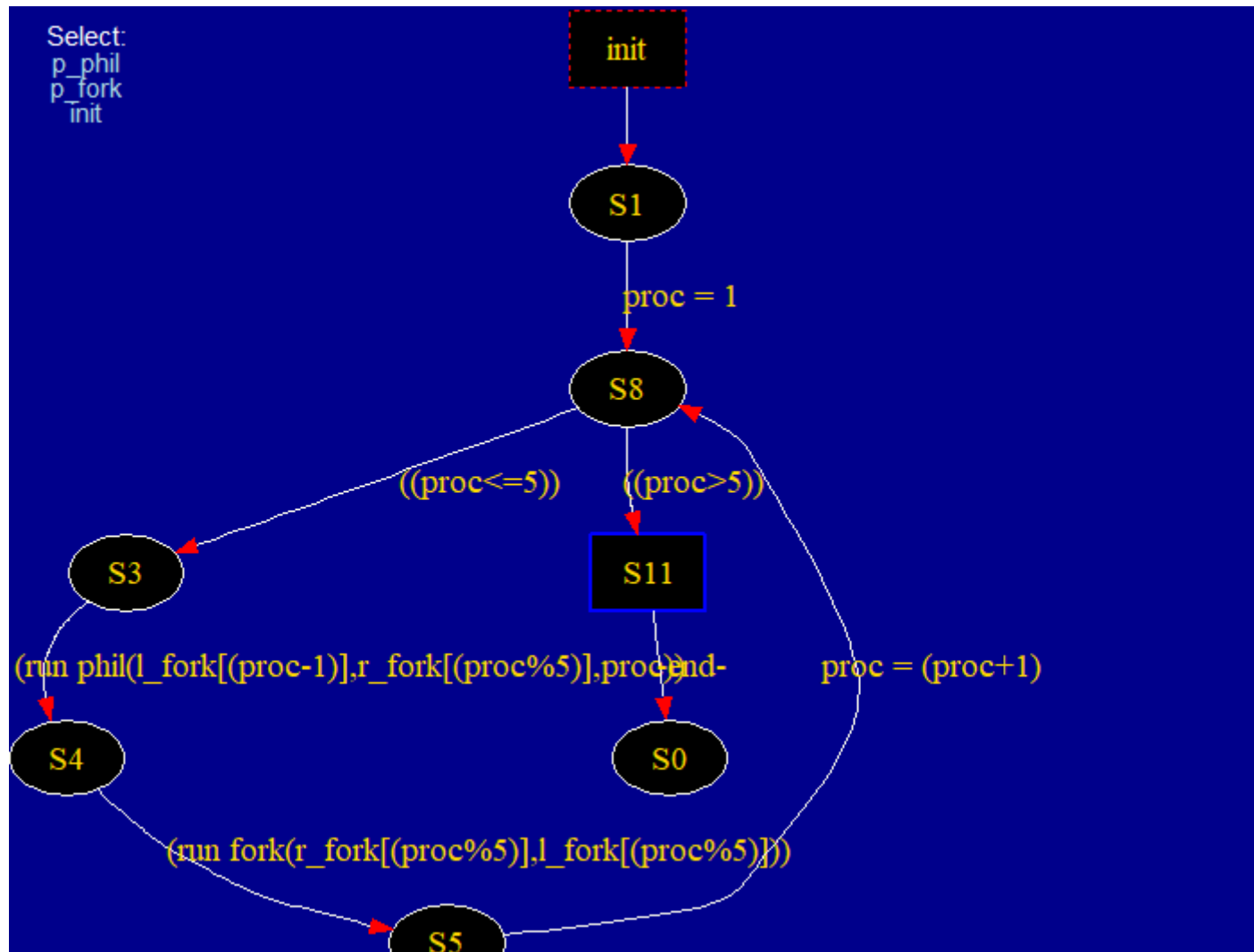
l_fork – канал для взаимодействия
философа proc с левой вилкой, номер
proc-1

r_fork – каналу для взаимодействия
философа proc с правой вилкой, номер
proc

Запуск начинается с процессов с
номером 1



Полувтомат процесса init



Проверка на возможность возникновения ситуации общего голодания

- Необходимо проверить свойство **Safety** в режиме верификации
- Во вкладке **Verification** в **Safety** должна стоять галочка **+invalid endstates (deadlock)**
- Затем в отчете также отражен **+** возле **invalid endstates**
- В режиме симуляции можно посмотреть траекторию (Mode: Guided with trail...) и проанализировать полученную траекторию

Моделирование задачи об обедающих философах с использованием разделяемых переменных

- оставляем только тип процесса «философ»
- создаем глобальную переменную – массив размерности N из элементов типа `bit`
- семантика состояний в процессе типа «философ» - прежняя
- если в глобальном массиве значение i -го элемента равно 1, то i -ая вилка занята

Возможные состояния философа

S_0 - философ размышляет. В этом состоянии он может проголодаться – тогда спонтанный переход в состояние S_1

S_1 - философ хочет есть, поэтому ему требуются вилки. Философ тянется за вилкой, лежащей слева от него. Если эта вилка отсутствует, то он в состоянии S_1 ждет ее освобождения. Если левая вилка свободна, то он ее берет и переходит в состояние S_2

S_2 - философ ожидает правую вилку. Если правая вилка отсутствует, то он будет находиться в этом состоянии до ее освобождения. Если правая вилка свободна, то он ее берет и переходит в состояние S_3

S_3 - философ ест.

S_4 - После того, как философ поел, он кладет на стол левую вилку, переходя в следующее состояние S_5 .

S_5 - философ кладет на стол правую вилку и возвращается к размышлениям (в состояние S_0)

Процессный тип «философ» при использовании разделяемых переменных

```
bit forks[N];
proctype phil (chan left, right; byte mn)
{
    byte cur_state = 0;
    do
        :: (cur_state == 1) -> (forks[left] == 0) ->
            atomic {forks[left] = 1; cur_state = 2 }
        :: (cur_state == 2) -> (forks[right] == 0) ->
            atomic {forks[right] = 1; cur_state = 3 }
        :: (cur_state == 3) -> cur_state = 4;
        :: (cur_state == 4) -> atomic {forks[right] = 0;
                                     cur_state = 5 }
        :: (cur_state == 5) -> atomic {forks[left] = 0;
                                     cur_state = 0 }
        :: (cur_state == 0) -> cur_state = 1;
    od
}
```

Процесс `init`

```
init
{
byte proc;
proc = 1;
do
:: proc <= N ->
    run phil (proc-1, (proc%N), proc);
    proc++;
:: proc > N -> break
od
}
```

Траектория, по которой возникает блокировка, приводит к тому, что `cur_state` у всех процессов-филосов принимают значение 2, т.е. каждый философ уже взял левую вилку и хочет взять правую вилку; при этом все вилки заняты, т.е.

```
forks[i] = 0, i = 1, N
```

ДЗ 3. Обедающие философы

1) Попробовать в SPIN один из вариантов моделирования задачи об обедающих философах, зафиксировать траекторию, по которой происходит блокировка (приложить скриншот)

2) * Предложить на языке PROMELA **решение проблемы обедающих философов, т.е. разработать модель поведения, в рамках которой ни один из философов не будет голодать (будет вечно чередовать прием пищи и размышления)**

При этом сохраняется постановка задачи:

Перед каждым философом стоит тарелка спагетти, а между каждой парой лежит вилка. Философ может либо есть, либо размышлять. Есть одно ограничение — философ может есть только тогда, когда одновременно держит в руках две вилки. Философ может взять одну из ближайших вилок со стола (если она доступна) или положить вилку на стол (если уже держит ее). Взятие и возвращение разных вилок являются отдельными действиями

Для решения данной задачи можно использовать, например:

- официант-семафор (дает разрешение на взятие вилки)
- иерархию ресурсов (вилки пронумерованы, определен порядок)

Показать средствами SPIN, что проблема обедающих философов действительно решена (не возникнет блокировок)